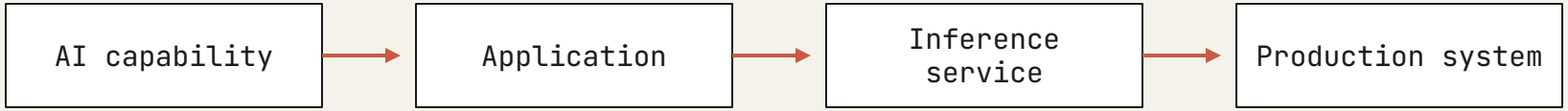
Using a model is one thing.
Operating it reliably is another.



Performance

Control

Scale

Model access is changing.

PROVIDER-MANAGED MODELS

- API access
- Provider operates infrastructure
- Limited runtime control

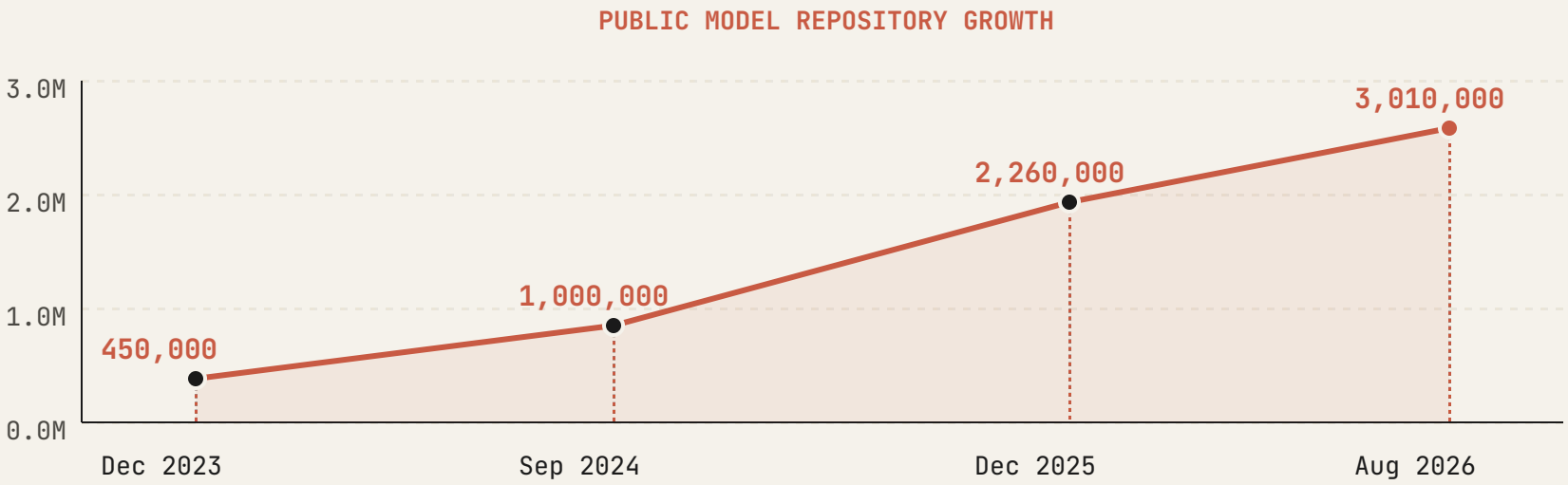
OPEN-WEIGHT MODELS

- Downloadable weights (license dependent)
- Run on private or rented infrastructure
- Greater deployment flexibility

SUMMARY TAKEAWAY

The deployment decision is becoming more flexible.

More models, derivatives, and deployment choices.



Repository counts indicate ecosystem size, not direct quality or active usage. Approximate figures.

More control also means more operational responsibility.

KEY DRIVERS

01 // CONTROL
Data governance

02 // ADAPTATION
Customization

03 // EFFICIENCY
Economics

04 // INFRASTRUCTURE
Deployment flexibility

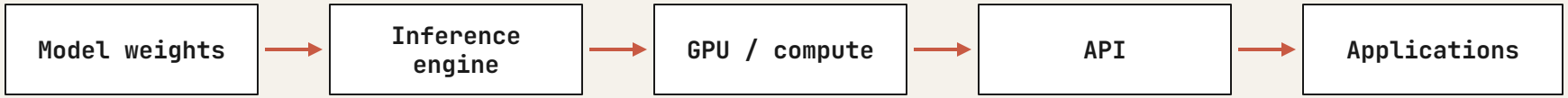
ENTERPRISE ADOPTION // MCKINSEY SURVEY FINDINGS

63% use open-source AI models.

76% expect usage to increase.

Next: run a model, inspect the GPU, and examine the serving stack.

ARCHITECTURE FLOW



KEY ENGINEERING QUESTIONS

01. How efficiently does the model run?
02. What happens as concurrent demand increases?
03. How do we measure performance and reliability?

Core Challenges in Model Execution

STAGE 01
Model Weights

→

STAGE 02 (CORE)
Inference Engine

→

STAGE 03
API / Application

PRIMARY SYSTEM BOTTLENECKS

- Weight execution mechanics
- Request processing architecture
- Token generation efficiency

"The engine dictates how models run, respond, and scale for multiple users."

Inference & Serving Engines

Developer-Friendly Local Inference

PRIMARY LOCAL ENGINE

Ollama

Streamlined local orchestration, model packaging, and CLI experience.

C/C++ BACKEND

llama.cpp

Minimalist, high-performance CPU & GPU edge execution runtime.

High-Performance Serving & Deployment

PRIMARY SERVING

vLLM

High-throughput LLM serving via PagedAttention memory management.

STRUCTURED EXECUTION

SGLang

Optimized runtime for complex prompt workflows & constraints.

NVIDIA HARDWARE

TensorRT-LLM

Maximized GPU acceleration & low-level kernel fusion.

ENTERPRISE READY

Hugging Face TGI

Production-grade text generation inference container setup.

Architecture & Capabilities




```
$ ollama run <model>
```
Execute & interact with local models

```
$ ollama ps
```
Inspect loaded models & hardware usage

- Model download and storage
- Templates and generation parameters
- Automatic device placement
- Local API

"Reduces friction between finding and experimenting with models."

vLLM: High-Throughput Serving Architecture



Core Innovations

- PagedAttention & efficient KV-cache memory management
- Continuous batching for iteration-level scheduling
- OpenAI-compatible API interface

24×

HIGHER THROUGHPUT

Compared to Hugging Face Transformers baseline in initial benchmarks.

Originated at UC Berkeley (2023)

"Focus shifts to serving requests efficiently at scale."

Serving Architecture Options

Ollama (Local)

Simple model acquisition

Straightforward local API

Ideal for prototyping

vLLM (Scale)

High-throughput serving

Continuous batching & KV-cache

Built for concurrent workloads

Tools overlap, but primary design emphases differ.

```
Next: Cloud GPU serving engine demonstration.
```

Inference Is a Systems Problem

vLLM System Execution Pipeline



Core Architectural Pillars

- **Request Scheduling**
  Coordinates incoming client requests and optimizes arrival queues for model processing.
- **Continuous Batching**
  Dynamically adds and removes iteration-level requests to maximize throughput.
- **Efficient KV-Cache**
  Manages key-value memory blocks cleanly to minimize internal fragmentation.

KEY ENGINEERING PRINCIPLE

High throughput means serving more inference work over time; it does not automatically mean the lowest latency for every user.

Historical Note: vLLM introduced PagedAttention in 2023. Its original evaluation reported 2–4× higher throughput at the same latency than FasterTransformer and Orca in the workloads tested.

(Source: Original PagedAttention paper, https://doi.org/10.1145/3600006.3613165) — *Benchmark results depend on model, workload, hardware, and configuration.*

Generation Needs Memory, Not Just Compute

01. Attention State Reuse

Stores intermediate attention states (Keys and Values) for instant reuse during auto-regressive generation.

02. Computation Reduction

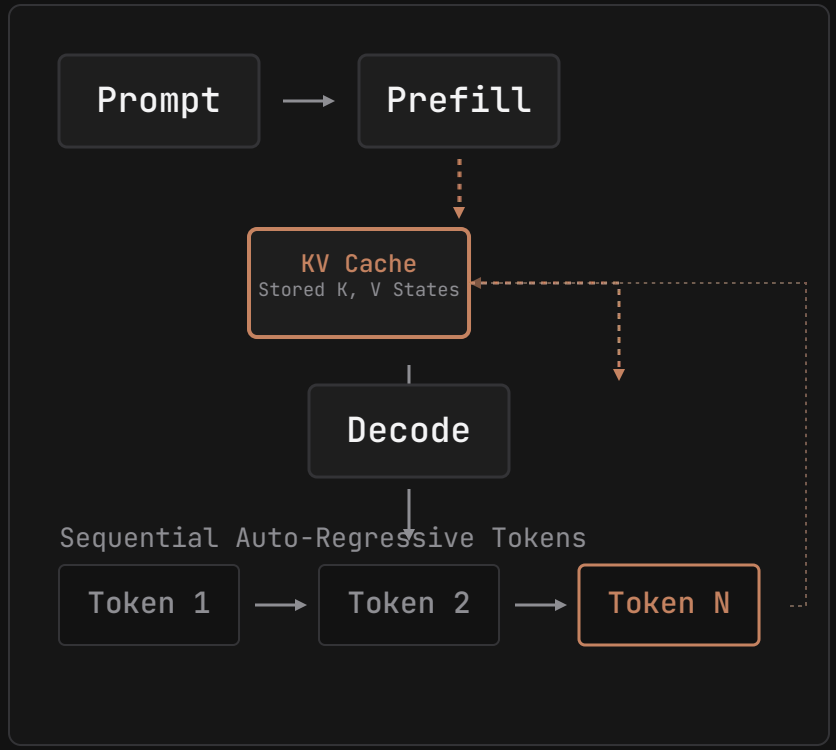
Significantly reduces the need to recompute attention states for all previous tokens at each sequential decoding step.

03. Dynamic Memory Footprint

Continuously consumes GPU memory as target sequence lengths grow and more requests are served concurrently.

KEY SYSTEM TAKEAWAY

Longer contexts and more active requests directly increase KV-cache memory demand.



Allocate Memory as It Is Needed

Virtual memory inspiration → Block-based KV-cache management

Illustrative Conventional Allocation

Request A (Short sequence)

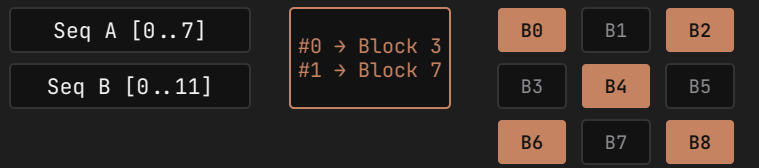
KV Content | Unused Space (Reserved)

Request B (Longer sequence)

KV Content (Occupies More) | Unused

Pre-allocates contiguous, over-reserved regions to fit max sequence length, creating heavy internal memory fragmentation and wasted capacity.

Paged KV Cache



Fixed-size physical KV-cache blocks mapped dynamically via a block table. Enables on-demand, non-contiguous physical allocation with near-zero wasted space.

KEY ENGINEERING TAKEAWAY

More efficient memory use allows significantly more active sequences to fit simultaneously in GPU memory.

Source: https://vllm.ai/blog/2023-06-20-vllm



1. TIME TO FIRST TOKEN (TTFT)
Latency before initial response token is generated (prefill phase).

2. INTER-TOKEN LATENCY
Time elapsed between each subsequent token (decode generation rate).

3. AGGREGATE TOKEN THROUGHPUT
Total output tokens generated per second across all active requests.

4. GPU & VRAM UTILIZATION
Percentage of compute cores & KV cache memory actively utilized.

LATENCY DEFINITION

How long a user waits for a response or the next token.

THROUGHPUT DEFINITION

How much inference work the server completes over time (output tokens / sec).

From Model Serving to Inference-Aware Routing

Application Client ⟶ K8s Gateway + llm-d Router ⟶ InferencePool (Model Servers) ⟶ vLLM Model Execution

Kubernetes

Deploys and manages the underlying infrastructure and workloads across clusters.

vLLM

Executes high-throughput LLM models and efficiently handles token generation.

llm-d

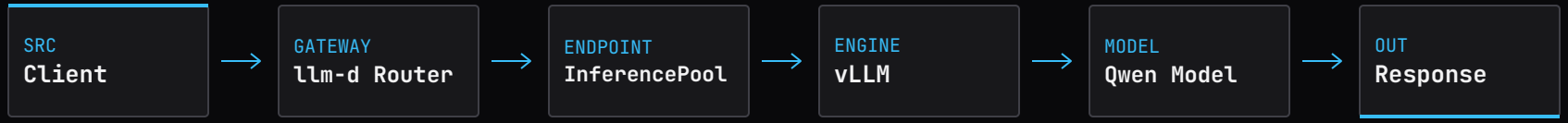
Provides inference-aware routing across eligible model-server endpoints using the Gateway API Inference Extension.

Ordinary load balancing simply distributes traffic. Inference-aware routing optimizes requests based on specific LLM workload metrics.

Tracing a Request Through the Stack

// SYSTEM ARCHITECTURE & INFERENCE PATHWAY

01 / REQUEST FLOW PIPELINE



02 / DEMONSTRATION STEPS

STEP 01 INSPECT

Inspect Kubernetes resources and running workloads.

STEP 02 REQUEST

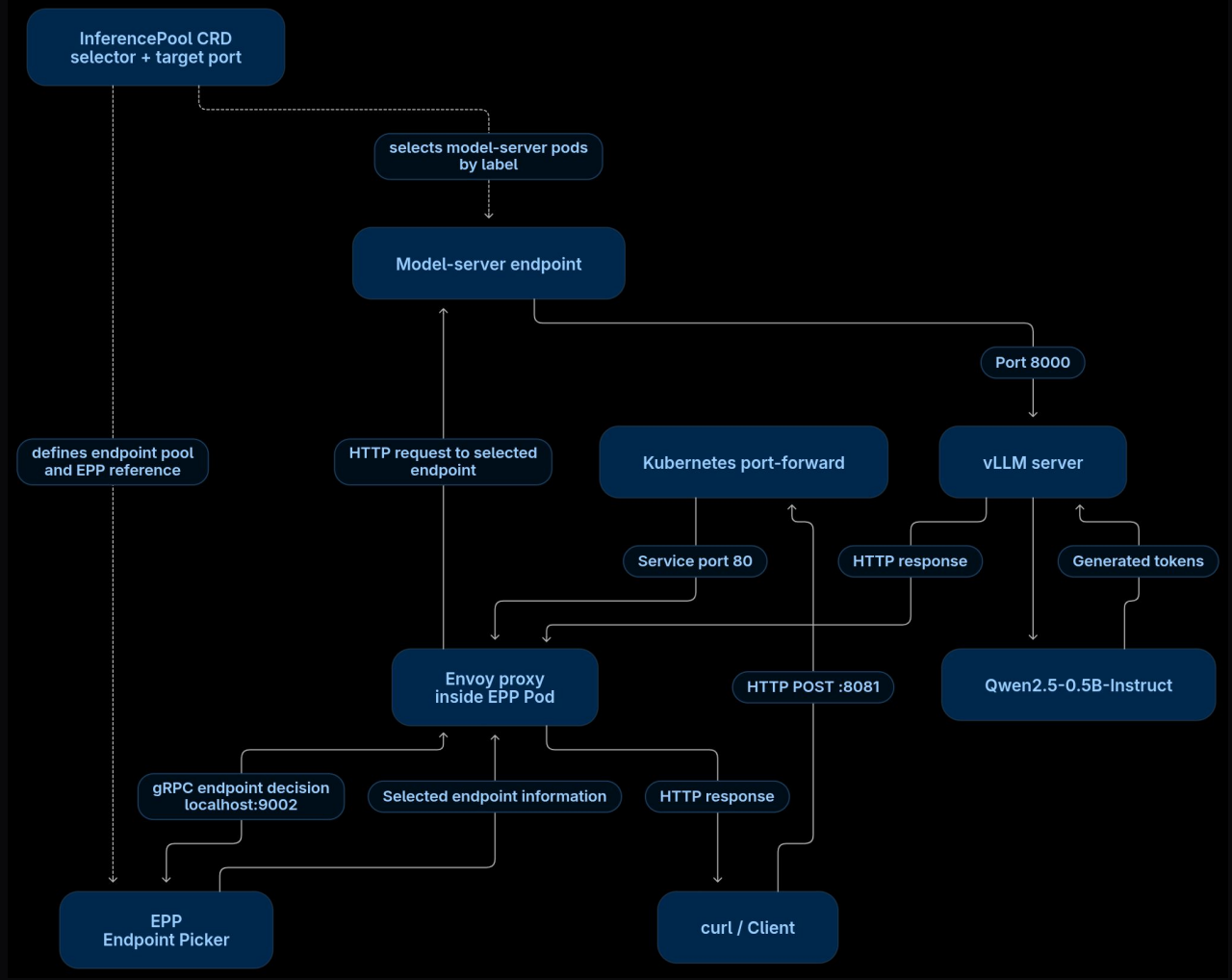
Send a chat-completion request through the serving endpoint.

STEP 03 OBSERVE

Observe component connections providing the inference API.

NOTE: CPU-BASED ARCHITECTURAL DEMONSTRATION · SINGLE MODEL-SERVER REPLICA

This demonstrates the architecture and request path, rather than production-scale performance improvements.


InferencePool CRD
selector + target port
selects model-server pods
by label
Model-server endpoint
Port 8000
defines endpoint pool
and EPP reference
HTTP request to selected
endpoint
Kubernetes port-forward
vLLM server
Service port 80
HTTP response
Generated tokens
Envoy proxy
inside EPP Pod
HTTP POST :8081
Qwen2.5-0.5B-Instruct
gRPC endpoint decision
localhost:9002
Selected endpoint information
HTTP response
EPP
Endpoint Picker
curl / Client

The AI Inference Stack

A scalable AI inference system operating across a 5-layer vertical architecture.

LAYER 1 **Application**	Chatbots, agents, or API clients initiating requests.
LAYER 2 **Inference Gateway**	`(llm-d)` Handles request routing and inference-aware endpoint selection.
LAYER 3 **Inference Engine**	vLLM, SGLang, or other supported model servers.
LAYER 4 **Model & Compute**	Manages model weights, CPU/GPU memory, KV cache, and accelerator execution.
FOUNDATION **Kubernetes Infra**	Manages Pods, Services, scheduling, networking, health checks, and resource allocation.

KEY TAKEAWAY // This layered approach separates infrastructure operations from inference execution and intelligent routing.